

Localização Indoor Baseada em Imagens do Teto

Estimativa de posição em ambientes fechados por reconhecimento visual do teto auxiliado por GPS aproximado

Munif Gebara · Projeto de pesquisa · Junho de 2026

Resumo

A localização de pessoas e ativos em ambientes fechados é um problema relevante e ainda mal resolvido: o GPS degrada-se sob coberturas e lajes, e soluções por Wi-Fi ou Bluetooth exigem infraestrutura dedicada e recalibração frequente. Este trabalho propõe e implementa um sistema que estima a posição do usuário sobre a planta baixa a partir de uma fotografia do teto combinada com uma leitura de GPS de baixa precisão. A hipótese central é que o teto é uma referência visual estável — pouca oclusão por pessoas, baixa variação temporal e iluminação relativamente constante — adequada ao reconhecimento visual de lugar. O sistema combina um filtro geográfico grosseiro, busca por similaridade de embeddings de imagem e verificação geométrica opcional, seguidos de interpolação ponderada da posição. Apresentamos a formalização do método, a arquitetura, a implementação (API em Python/FastAPI, PostgreSQL com pgvector e earthdistance, e três aplicativos web progressivos) e uma avaliação em cenário sintético, na qual o sistema obteve erro mediano de aproximadamente 0,6 m e 100% de acertos dentro de 2 m. Discutimos limitações e delineamos a avaliação prática com dados reais como principal trabalho futuro.

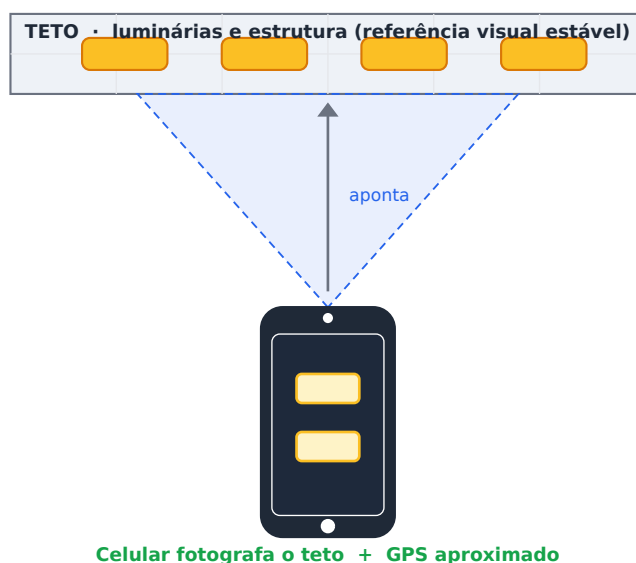
Palavras-chave: localização indoor; reconhecimento visual de lugar; embeddings de imagem; pgvector; PostgreSQL; FastAPI.

1. Introdução

Sistemas de navegação por satélite (GNSS) revolucionaram a localização em ambientes abertos, mas sua precisão cai drasticamente em ambientes internos, onde o sinal é atenuado e sofre múltiplos caminhos. Em shopping centers, aeroportos e hospitais, isso inviabiliza orientação ao usuário, logística e análise de fluxo. Abordagens por rádio (Wi-Fi fingerprinting, balizas Bluetooth, UWB) alcançam boa precisão, mas dependem de infraestrutura instalada e recalibração periódica, elevando custo e manutenção.

A visão computacional é uma alternativa atraente: o ambiente já é rico em informação visual e o smartphone do usuário dispõe de câmera e GPS. O reconhecimento visual de lugar (Visual Place Recognition, VPR) identifica onde uma imagem foi capturada comparando-a com uma base de referência. Fotos ao nível dos olhos, porém, sofrem com oclusão por pessoas, mudanças de vitrines e variações de iluminação.

Este trabalho explora usar o **teto** como referência. Tetos são visualmente estáveis, raramente ocluídos por pessoas e têm iluminação mais uniforme, tornando-os âncoras visuais confiáveis. A ideia, ilustrada na Figura 1, é simples: o usuário aponta o celular para cima, fotografa o teto e fornece um GPS aproximado; o sistema devolve a posição na planta baixa.



Celular fotografa o teto + GPS aproximado

Figura 1. Conceito: o celular é apontado para cima e fotografa o teto (luminárias e estrutura); a foto, com o GPS aproximado, alimenta a estimativa de posição.

A Figura 2 mostra a arquitetura geral do sistema.

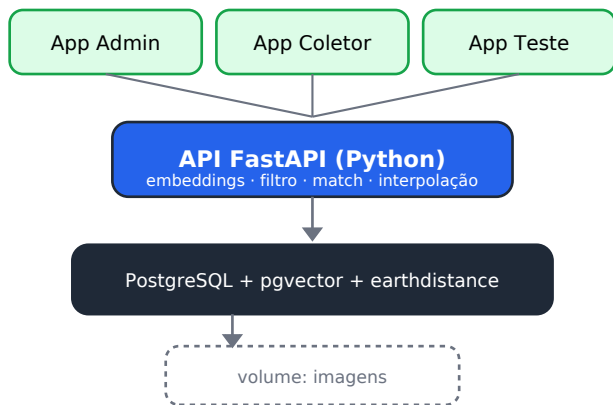


Figura 2. Arquitetura: três PWAs consomem a API, que persiste dados e embeddings no PostgreSQL e as imagens em volume.

2. Hipótese e objetivos

Hipótese. Dada uma planta baixa e uma base de fotografias do teto rotuladas com a posição em que foram tiradas, é possível estimar a posição de uma nova fotografia do teto sobre a planta, usando o GPS apenas como filtro grosseiro e a imagem como discriminador fino. A Figura 3 resume as duas fases de operação.

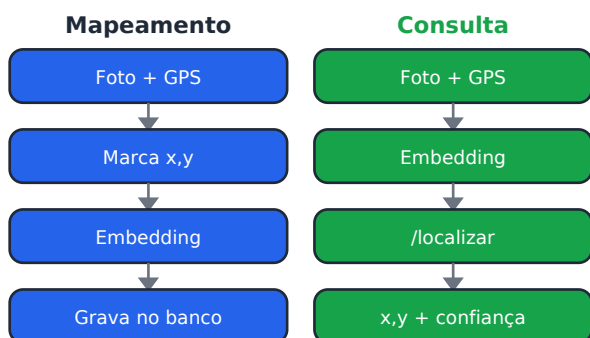


Figura 3. As duas fases: mapeamento (coleta rotulada) e consulta (estimativa da posição).

Objetivos:

- Implementar uma API que receba foto do teto + GPS e devolva (x, y) na planta com confiança.
- Construir o fluxo de mapeamento e de consulta em aplicativos web.
- Definir e medir uma métrica de erro em metros, validando o método.
- Estabelecer base extensível para modelos de visão mais fortes.

3. Trabalhos relacionados

Descritores visuais. O reconhecimento de lugar evoluiu de descritores locais clássicos — SIFT [1] e ORB [2], casados e filtrados por RANSAC [3] — para descritores globais aprendidos, como NetVLAD [4], e, recentemente, para características auto-supervisionadas de propósito geral como

DINOv2 [5]. Para verificação geométrica, SuperPoint [6] e LightGlue [7] superam pipelines clássicos. O descritor de imagem reduzida (tiny image) [8] é uma linha de base simples e eficiente para recuperação.

Localização indoor. Soluções por rádio (Wi-Fi fingerprinting, BLE, UWB) são precisas, mas dependem de infraestrutura e calibração. Abordagens visuais dispensam hardware adicional; o uso específico do teto, menos sujeito a oclusão e mudanças, é comparativamente pouco explorado e é o foco deste trabalho.

Sistemas. A extensão pgvector [9] habilita busca aproximada por vizinhos em PostgreSQL, e a extensão earthdistance permite consultas geográficas por raio sem a complexidade do PostGIS. Combinamos essas peças num pipeline híbrido específico para o teto.

4. Arquitetura e método

4.1 Pipeline de localização

A consulta percorre quatro etapas encadeadas (Figura 4): filtro geográfico, busca por similaridade, verificação geométrica opcional e interpolação da posição.



Figura 4. Pipeline de localização: do par foto+GPS à posição estimada.

4.2 Modelo de dados

Três entidades (Figura 5): **local** (o ambiente), **planta** (SVG com escala em metros por unidade) e **foto** (imagem do teto com GPS, posição na planta, embedding e tipo). Um índice GiST sobre `ll_to_earth(lat, lon)` sustenta o filtro por raio e um índice HNSW sobre o embedding sustenta a busca por similaridade.

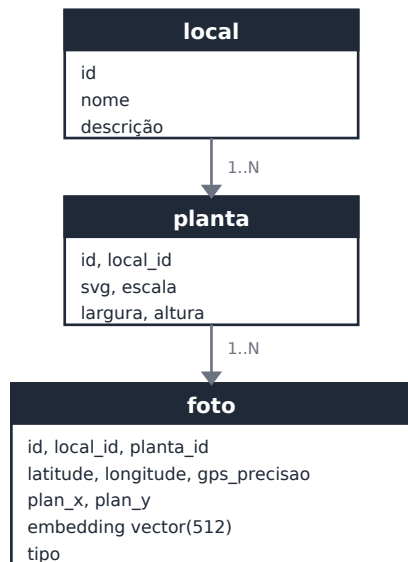


Figura 5. Modelo de dados e cardinalidades.

4.3 Formalização

Seja I uma imagem do teto. O embedding $e \in \mathbb{R}^{512}$ é obtido convertendo I para tons de cinza, redimensionando para 32×16 , achatando no vetor v e normalizando:

$$e = (v - \mu) / \|v - \mu\|_2, v \in \mathbb{R}^{512}$$

onde μ é a média de v (invariância a brilho). Como os embeddings são normalizados, a distância de cosseno e a similaridade entre a consulta q e uma referência i são:

$$d(e_q, e_i) = 1 - e_q \cdot e_i, s_i = 1 - d(e_q, e_i)$$

O conjunto de candidatos C é restrito por proximidade geográfica, com raio r dependente da imprecisão do GPS:

$$C = \{ i : \text{tipo}_i = \text{mapeamento} \wedge \text{dist}_{\text{geo}}(g_q, g_i) \leq r \}$$

Cada candidato recebe um peso que cresce com a similaridade e , quando há verificação geométrica, com o número de inliers:

$$w_i = \max(s_i, 0)^2 \cdot (1 + \text{inliers}_i)$$

A posição é a média ponderada das posições dos candidatos:

$$(\hat{x}, \hat{y}) = (\sum_i w_i x_i / \sum_i w_i, \sum_i w_i y_i / \sum_i w_i)$$

e o erro de localização, em metros, usa a escala da planta:

$$\varepsilon = \|(\hat{x}, \hat{y}) - (x, y)\|_2 \cdot \text{escala}$$

4.4 Representação e filtro geográfico

O backend padrão é o descritor tiny-image acima, que mantém o serviço leve (sem dependências pesadas) e valida o pipeline de ponta a ponta; a arquitetura permite trocá-lo por um backend forte (p. ex., DINOv2) sem alterar o restante. Como o PostgreSQL disponível não inclui PostGIS, o filtro por raio usa cube/earthdistance: ll_to_earth projeta a

coordenada em um ponto 3D e earth_box/earth_distance realizam a busca por proximidade com índice GiST.

5. Implementação

A API usa **FastAPI** (Python), persistência em **PostgreSQL 17** (pgvector) com as extensões vector, cube e earthdistance, e migrações com Alembic; as imagens ficam em volume persistente. O front-end são três PWAs em **Vue 3 + Vite** (admin, coletor e teste) que compartilham módulos de câmera, GPS e cliente HTTP. A implantação é em **Kubernetes (k3s)** com Ingress Traefik; a integração contínua (GitHub Actions) executa estilo e testes a cada alteração, e o desenvolvimento seguiu o fluxo issue → ramo → Pull Request → integração.

6. Avaliação experimental

Realizamos uma avaliação controlada e reproduzível para validar o pipeline e o método de medição antes da coleta de dados reais. Cria-se um local com planta de 1000×1000 unidades e escala de 0,05 m/unidade (ambiente de ~50 m). Mapeiam-se 25 posições em grade, cada uma com um padrão visual distinto. Para cada posição, gera-se uma consulta a partir da mesma imagem perturbada com ruído gaussiano ($\sigma = 18$) e variação de brilho (+20), simulando recaptura. O erro é a distância à posição verdadeira, convertida em metros pela escala (Tabela 1, Figura 6).

Métrica	Unid.	Metros
Erro mediano	11,90	0,595
Erro médio	10,76	0,538
Erro máximo	18,93	0,947
Acerto (≤ 2 m)	—	100%

Tabela 1. Resultados da avaliação sintética ($n = 25$, tiny-image).

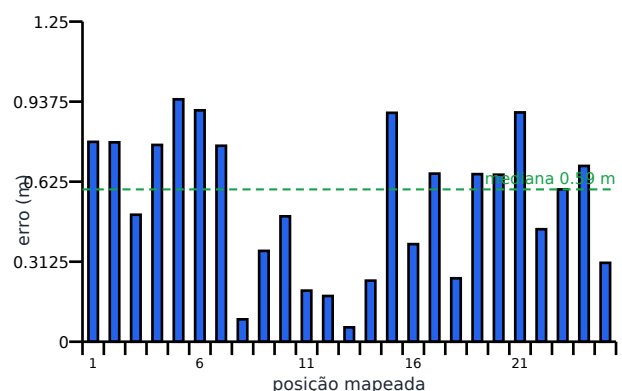


Figura 6. Erro de localização por posição mapeada; todas abaixo do limiar de 2 m.

O sistema localizou corretamente todas as consultas dentro de 2 m, com erro mediano de cerca de 0,6 m. O resíduo decorre da interpolação ponderada, que

mistura o candidato correto com vizinhos de baixa semelhança. O objetivo é verificar a corretude do pipeline e da medição; os números não devem ser extrapolados para condições reais.

[8] Torralba, A.; Fergus, R.; Freeman, W. T. 80 million tiny images. IEEE TPAMI, 2008.

[9] pgvector: vector similarity search for PostgreSQL. github.com/pgvector/pgvector.

7. Discussão e limitações

- **Backend de embedding.** O tiny-image é sensível a rotação, escala e iluminação; padrões de teto repetitivos podem confundi-lo.
- **Avaliação sintética.** Valida o método, não a precisão em campo; não captura variação de pose da câmera nem mudanças reais do ambiente.
- **GPS indoor.** O filtro pressupõe GPS minimamente informativo; sem sinal, o raio precisa ser ampliado.
- **Georreferência.** GPS e coordenadas da planta são, por ora, independentes; a conversão entre eles fica para fase posterior.

8. Trabalhos futuros

- **Avaliação prática com dados reais** em um ambiente real, com posição de referência conhecida, medindo erro em metros sob variação de horário e de aparelho.
- **Embeddings fortes** (DINOv2, NetVLAD) comparados em precisão, latência e custo.
- **Verificação geométrica moderna** (SuperPoint + LightGlue) versus ORB/RANSAC.
- **Calibração** de raio, top-N e limiares por local.
- **Georreferência da planta** com pontos de controle (fase 2).
- **Operação:** exposição externa segura, monitoramento e reindexação incremental.

9. Conclusão

Apresentamos um sistema completo e funcional de localização indoor baseado em imagens do teto, do modelo de dados à API, aos aplicativos e à implantação em Kubernetes. A avaliação confirmou a corretude do pipeline e do método de medição, com erro mediano sub-métrico em cenário controlado, sustentando a hipótese de que o teto é uma referência visual conveniente e motivando a avaliação prática com dados reais.

Referências

- [1] Lowe, D. G. Distinctive image features from scale-invariant keypoints. IJCV, 2004.
- [2] Rublee, E. et al. ORB: An efficient alternative to SIFT or SURF. ICCV, 2011.
- [3] Fischler, M. A.; Bolles, R. C. Random sample consensus. Communications of the ACM, 1981.
- [4] Arandjelović, R. et al. NetVLAD: CNN architecture for weakly supervised place recognition. CVPR, 2016.
- [5] Oquab, M. et al. DINOv2: Learning robust visual features without supervision. arXiv:2304.07193, 2023.
- [6] DeTone, D.; Malisiewicz, T.; Rabinovich, A. SuperPoint: Self-supervised interest point detection and description. CVPR Workshops, 2018.
- [7] Lindenberger, P.; Sarlin, P.-E.; Pollefeys, M. LightGlue: Local feature matching at light speed. ICCV, 2023.